


Figure 1: Playing with the 'Disk on RAM' driver

that are to be used by VFS.

- Block drivers are designed to be integrated with the Linux buffer cache mechanism for efficient data access. Character drivers are pass-through drivers, accessing the hardware directly.

And these cause the implementation differences. Let's analyse the key code snippets from *ram_block.c*, starting at the driver's constructor *rb_init()*.

The first step is to register for an 8-bit (block) major number (which implicitly means registering for all 256 8-bit minor numbers associated with it). The function for that is as follows:

```
int register_blkdev(unsigned int major, const char *name);
```

Here, *major* is the major number to be registered, and *name* is a registration label displayed under the kernel window */proc/devices*. Interestingly, register_blkdev() tries to allocate & register a freely available major number, when 0 is passed for its first parameter major; on success, the allocated major number is returned. The corresponding de-registration function is as follows:

```
void unregister_blkdev(unsigned int major, const char *name);
```

Both these are prototyped in *<linux/fs.h>*.

The second step is to provide the device file operations, through the struct *block_device_operations* (prototyped in *<linux/blkdev.h>)* for the registered major number device files. However, these operations are too few compared to the character device file operations, and mostly insignificant. To elaborate, there are no operations even to read and write, which is surprising. But as we already know that block drivers need to integrate with the I/O schedulers, the read-write implementation is achieved through something called request queues. So, along with providing the device file operations, the following need to be provided:

- The request queue for queuing the read/write requests
- The spin lock associated with the request queue to protect its concurrent access

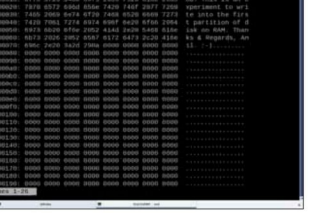



Figure 2: xxd showing the initial data on the first partition (/dev/rb1)

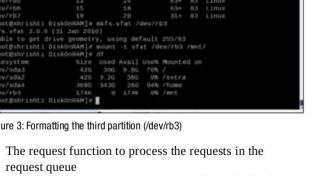



Figure 3: Formatting the third partition (/dev/rb3)

- The request function to process the requests in the request queue

Also, there is no separate interface for block device file creations, so the following are also provided:

- The device file name prefix, commonly referred to as *disk_name* (*"rb"* in the *'dor'* driver)
- The starting minor number for the device files, commonly referred to as *first_minor*.

Finally, two block-device-specific things are also provided, namely:

- The maximum number of partitions supported for this block device, by specifying the total minors.
- The underlying device size in units of 512-byte sectors, for the logical block access abstraction.

All these are registered through the struct *gendisk* using the following function:

```
void add_disk(struct gendisk *disk);
```

The corresponding delete function is as follows:

```
void del_gendisk(struct gendisk *disk);
```